Inductive bias meta-learning with generative models

Anna Novokshonova
Consultant: Fedor Sobolevsky, Muhammadsharif Nabiev
Expert: Oleg Bakhteev, PhD

Course: My first scientific paper
Intelligent Systems Department, MIPT

2026

Goal of research

Inductive bias is a model's preference for certain types of functions or data structures over others, expressed through better generalization performance.

Goal

Extraction of inductive bias of differentiable learning algorithms.

Problem

The existing solution assumes prior knowledge of the data distribution or requires a fixed dataset.

Solution

A meta-learning algorithm that trains a generator to produce datasets that are easy to generalize to the target model and that reflect its inductive bias.

Extracting inductive bias via generating easy-to-generalize datasets

Previous work[a]

Meta-learning a function that assigns labels to samples from a fixed dataset or data distribution.

Our extension

Meta-learning a generative model to produce entire synthetic datasets.

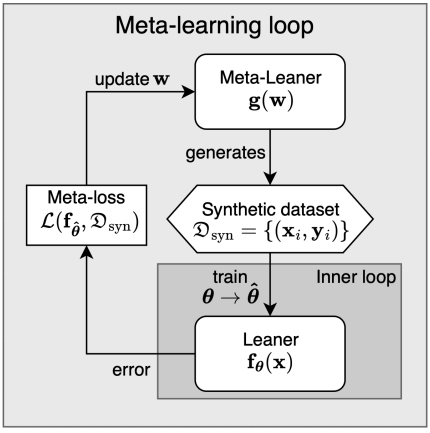

Figure: Meta-learning scheme

[a] William Dorrell, Maria Yuffa, and Peter Latham. *Meta-Learning the Inductive Biases of Simple Neural Circuits*. 2023. arXiv: 2211.13544 [q-bio.NC]. URL: https://arxiv.org/abs/2211.13544.

Problem statement

Given a target model $\mathbf{f}_{\boldsymbol{\theta}} : \mathbb{X} \to \mathbb{Y}$ with loss function $\ell(\mathbf{f}_{\boldsymbol{\theta}}(\mathbf{x}), \mathbf{y})$, the generator $\mathbf{g}(\mathbf{w})$ defines a parametric family of conditional distributions $p(\mathbf{x} \mid \mathbf{y}, \mathbf{w})$ over inputs $\mathbf{x}$ given labels $\mathbf{y}$.

Required to find data distribution function p^*, corresponding to parameters $\mathbf{w}^*$, the solution to the following optimization problem:

$$\mathbf{w}^* = \arg\min_{\mathbf{w}} \; \mathbb{E}_{\mathcal{D}_{\text{syn}} \sim p(\mathbf{x}|\mathbf{y},\mathbf{w})} \left[\mathcal{L}\left(\mathbf{f}_{\hat{\boldsymbol{\theta}}}, \mathcal{D}_{\text{syn}} \right) \right] \quad \text{s.t.} \quad |\mathfrak{D}_{\text{train}} \cup \mathfrak{D}_{\text{test}}| = d$$

where $\hat{\boldsymbol{\theta}} = \arg\min_{\boldsymbol{\theta} \in \Theta} \sum_{(\mathbf{x},\mathbf{y}) \in \mathfrak{D}_{\text{train}}} \ell(\mathbf{f}_{\boldsymbol{\theta}}(\mathbf{x}), \mathbf{y})$ and $\mathcal{L}$ is meta loss function.

Meta loss function

The meta-loss function $\mathcal{L}$ combines prediction error $\mathcal{L}_g$ and regularization $\mathcal{R}$:

$$\mathcal{L} = \mathcal{L}_g + \lambda \mathcal{R}$$

Prediction error $\mathcal{L}_g$ (MSE or cross-entropy):

$$\mathcal{L}_{\text{CE}} = -\frac{1}{|\mathfrak{D}_{\text{test}}|} \sum_{(\mathbf{x},\mathbf{y}) \in \mathfrak{D}_{\text{test}}} \sum_{i=1}^{m} y_i \log \left([\mathbf{f}_{\hat{\boldsymbol{\theta}}}(\mathbf{x})]_i \right)$$

$$\mathcal{L}_{\text{MSE}} = \frac{1}{|\mathfrak{D}_{\text{test}}|} \sum_{(\mathbf{x},\mathbf{y}) \in \mathfrak{D}_{\text{test}}} \| \mathbf{y} - \mathbf{f}_{\hat{\boldsymbol{\theta}}}(\mathbf{x}) \|^2$$

Regularization $\mathcal{R}$:

$$\mathcal{R}_{\text{dist}} = \frac{1}{C} \sum_{k=1}^{C} \frac{1}{n_k(n_k-1)} \sum_{\substack{i,j \in \mathcal{I}_k \\ i \neq j}} \| \mathbf{x}_i - \mathbf{x}_j \|_2^2, \quad \mathcal{R}_{\text{SVD}} = \frac{1}{r} \sum_{k=1}^{r} \big(\max(\tau - \sigma_k, 0) \big)^2$$

Solution

Algorithm 1 Meta-learning for inductive bias extraction

1: Initialize generator $\mathbf{g}(\mathbf{w})$ with random parameters $\mathbf{w}$
2: **while** step $<$ total steps **do**
3: Generate synthetic dataset $\mathfrak{D}_{\text{syn}} = \{(\mathbf{x}_i, \mathbf{y}_i)\}$ with C classes and N samples per class
4: Split $\mathfrak{D}_{\text{syn}}$ into training set $\mathfrak{D}_{\text{train}}$ and test set $\mathfrak{D}_{\text{test}}$
5: Re-initialize target model $\mathbf{f}_{\boldsymbol{\theta}}$ from scratch
6: Train $\mathbf{f}_{\boldsymbol{\theta}}$ on $\mathfrak{D}_{\text{train}}$ in the inner loop $\rightarrow$ trained model $\mathbf{f}_{\hat{\boldsymbol{\theta}}}$
7: Compute meta-loss: $\mathcal{L}_{\text{meta}} = \mathcal{L}_g(\mathbf{f}_{\hat{\boldsymbol{\theta}}}, \mathfrak{D}_{\text{test}}) + \lambda \mathcal{R}(\mathfrak{D}_{\text{syn}})$
8: Take $\mathbf{w}$ gradient step on meta-loss
9: **end while**

CNN and 1x4 images

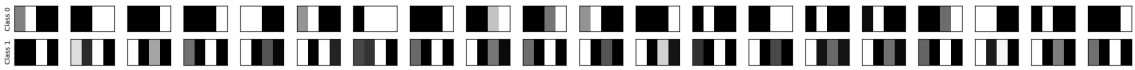

Figure: Obtained dataset for average intra-class distance regularization

Generator: Trainable tensor.
CNN Architecture: Conv1d with kernel 1×2, stride (1,2) $\rightarrow$ max pooling $\rightarrow$ linear layer $\rightarrow$ class logits.

Training	Test	Accuracy (%)
Original	Original	98.78 ± 8.82
Original	Swapped	98.78 ± 8.82
Swapped	Swapped	99.08 ± 7.25
Swapped	Original	99.08 ± 7.25

Table: Accuracy comparison on original and swapped data

Parameters:

1. 2 classes, 20 samples per class
2. 7 steps in inner loop
3. Learning rates: inner $= 0.5$, outer $= 0.05$

Meta-loss function: $\mathcal{L} = \mathcal{L}_{MSE} + 0.5 \cdot \mathcal{R}_{dist}$

CNN and 4x4 images: locality

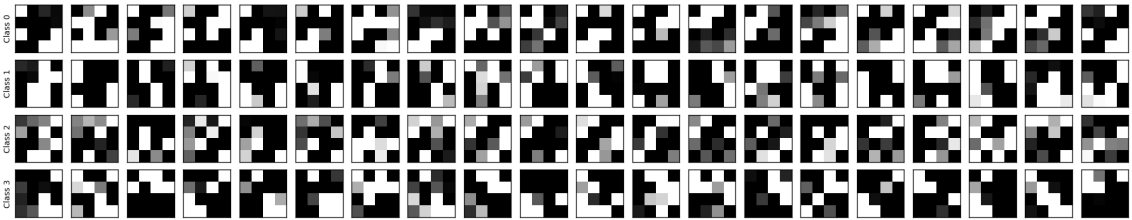

Figure: Obtained dataset for cross-entropy and average intra-class distance regularization

Generator: Trainable tensor.
CNN Architecture: Conv2d with kernel 2×2, stride 2 → ReLU → max pooling (2×2) → flatten → linear layer → class logits.
Parameters: 4 classes, 20 samples per class, 10 steps in inner loop, learning rates of 0.5 (inner) and 0.01 (outer).
Meta-loss function: $\mathcal{L}_{CE}$ / $\mathcal{L}_{MSE} + \mathcal{R}_{dist}$ / $\mathcal{R}_{SVD}$ (different combinations).

CNN and 4x4 images: locality

Setup	Original	Seed 42	Seed 123	Seed 456	Seed 789
CE+Dist	98.87 ± 5.33	98.87 ± 5.33	98.87 ± 5.33	98.87 ± 5.33	98.87 ± 5.33
MSE+Dist	99.03 ± 5.22	99.03 ± 5.22	99.03 ± 5.22	99.03 ± 5.22	99.03 ± 5.22
CE+SVD	99.33 ± 4.62	99.33 ± 4.62	99.33 ± 4.62	99.33 ± 4.62	99.33 ± 4.62
MSE+SVD	98.96 ± 5.46	98.96 ± 5.46	98.96 ± 5.46	98.96 ± 5.46	98.96 ± 5.46
MLP	62.24 ± 14.73	78.76 ± 13.02	77.41 ± 12.97	77.61 ± 12.97	76.05 ± 13.68

Table: Accuracy on original and shuffled data for all configurations

CNN and 4x4 images: translation invariance

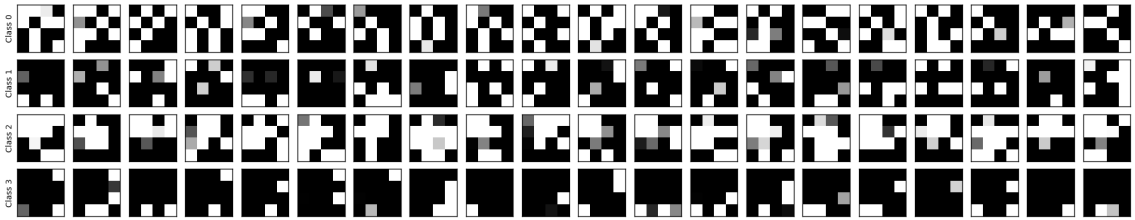

Figure: Obtained dataset

Shift type	**Accuracy (%)**
Original	98.78 ± 8.82
Shift horizontal (1px)	95.23 ± 10.12
Shift vertical (1px)	94.56 ± 11.34
Shift horizontal+vertical (1px)	89.12 ± 14.56

Table: Accuracy on original and shifted images

CNN Architecture: Conv2d with kernel 2×2, stride 1 → ReLU → max pooling (2×2) → flatten → linear layer → class logits.

Meta-loss function: $\mathcal{L} = \mathcal{L}_{MSE} + \mathcal{R}_{dist}$

RNN

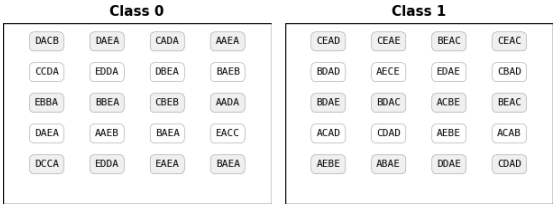

Figure: Obtained dataset

Generator: Trainable tensor.

RNN Architecture: RNN with hidden size 32, 2 layers $\rightarrow$ last hidden state $\rightarrow$ linear layer $\rightarrow$ class logits.

Parameters:

1. 2 classes, 20 samples per class
2. 10 steps in inner loop
3. Learning rates: inner $= 0.5$, outer $= 0.005$

Train data	**Test: Original**	**Test: Reversed**	**Test: Permuted**
Original	100.00 ± 0.00	0.00 ± 0.00	50.12 ± 17.01
Reversed	0.05 ± 0.79	99.80 ± 3.57	49.39 ± 17.25
Permuted	38.88 ± 22.07	37.27 ± 19.59	38.42 ± 14.98

Table: Accuracy on original, reversed and permuted test sets for models trained on different data types

Conclusion

1) We consider the problem of extracting the inductive bias of differentiable learning algorithms without prior knowledge of the input data distribution.
2) We propose a generative meta-learning framework where a generative model is trained to produce synthetic easy-to-generalize datasets that reflect the target model's inductive bias.
3) Experiments on CNNs and RNNs demonstrate that the generated datasets reflect the well-known inductive biases of these architectures: locality and translation invariance for CNNs, and sequential structure for RNNs.
4) Our method successfully extracts inductive bias without requiring any external data, offering a tool for analyzing different model architectures.